

ARTEFATOS DO PROJETO DE SOFTWARE - CRIAÇÃO DE CAMARÃO EM CATIVEIRO COM MONITORAMENTO IOT PARA GASTRONOMIA

SUMÁRIO

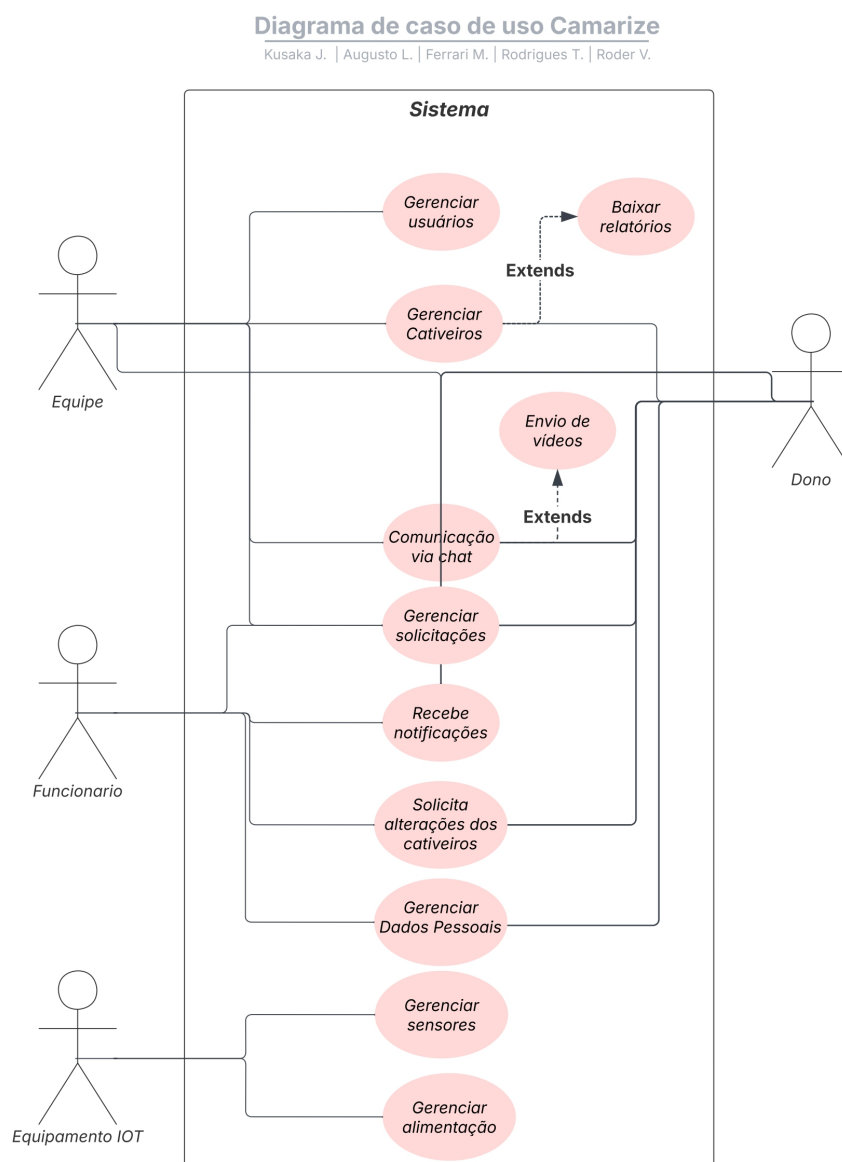
DIAGRAMA DE CASO DE USO	3
DIAGRAMA DE CLASSE	4
DIAGRAMA DE OBJETOS	5
DIAGRAMA DE REDES	7
MODELO DE NEGÓCIOS CANVAS	8
MODELO LÓGICO E CONCEITUAL DO BANCO DE DADOS	9
ANÁLISE DE RECORRÊNCIA DO ALGORITMO MERGE SORT	12
APLICAÇÃO IOT	14
ANÁLISE SWOT	22
DIÁRIO DE BORDO	22

DIAGRAMAS UML

Nesta seção serão apresentados os diagramas da UML utilizados para a modelagem do sistema desenvolvido. Dentre os diagramas utilizados, pode-se citar: Diagrama de Caso de Uso e Diagrama de Redes.

DIAGRAMA DE CASO DE USO

Figura 1 – Diagrama de caso de uso



Fonte: Autoria Própria

O Diagrama de Caso de Uso foi desenvolvido para demonstrar as funcionalidades que cada

ator possui no sistema. O dono/funcionário irá fornecer ao sistema suas informações para realizar o cadastro/login e o sistema deverá fornecer a tela inicial. Dentro do sistema, o dono poderá visualizar as informações sobre os cativeiros de maneira geral ou individual, aceitar solicitações enviadas pelo funcionário e utilizar o chat para conversar com a equipe sobre problemas.

Enquanto o dono/funcionário utilizar o sistema, ele receberá notificações a respeito dos tanques e dos camarões, verificar informações a fazenda e de seu perfil. Os equipamentos de monitoramento irão transmitir os dados de cada sensor e receberão comandos. O dispensador de ração irá receber o comando para liberar a ração para os camarões.

DIAGRAMA DE CLASSE

O diagrama de classes define a estrutura e os relacionamentos essenciais para o monitoramento e a gestão de cativeiros de camarões, visando otimizar as condições de criação. Ele contempla classes que representam os diferentes componentes do sistema, suas responsabilidades e as associações necessárias para uma operação integrada.

Entre as principais classes, `TiposCamaroes` identifica as espécies de camarões disponíveis, possibilitando categorizar diferentes tipos por meio de um identificador único e nome. `CondicoesIdeais` associa os tipos de camarões às condições ambientais ideais, como temperatura, pH e amônia, garantindo recomendações específicas para cada espécie. `EspecifCamarao` conecta uma espécie de camarão às suas Dietas e Condições Ideais, detalhando especificações específicas.

`Cativeiros` representa os espaços de criação, contendo informações como nome, data de instalação, foto, e as médias diárias de temperatura, pH e amônia, além de estar associados aos tipos de camarões por meio de uma relação de muitos para um `fk_id_tiposCamaroes`. Cada cativeiro também pode ser monitorado por sensores, que são gerenciados pela classe `Sensores` associados aos cativeiros via a classe `SensoresXCativeiros`, permitindo a captura de dados ambientais em tempo real.

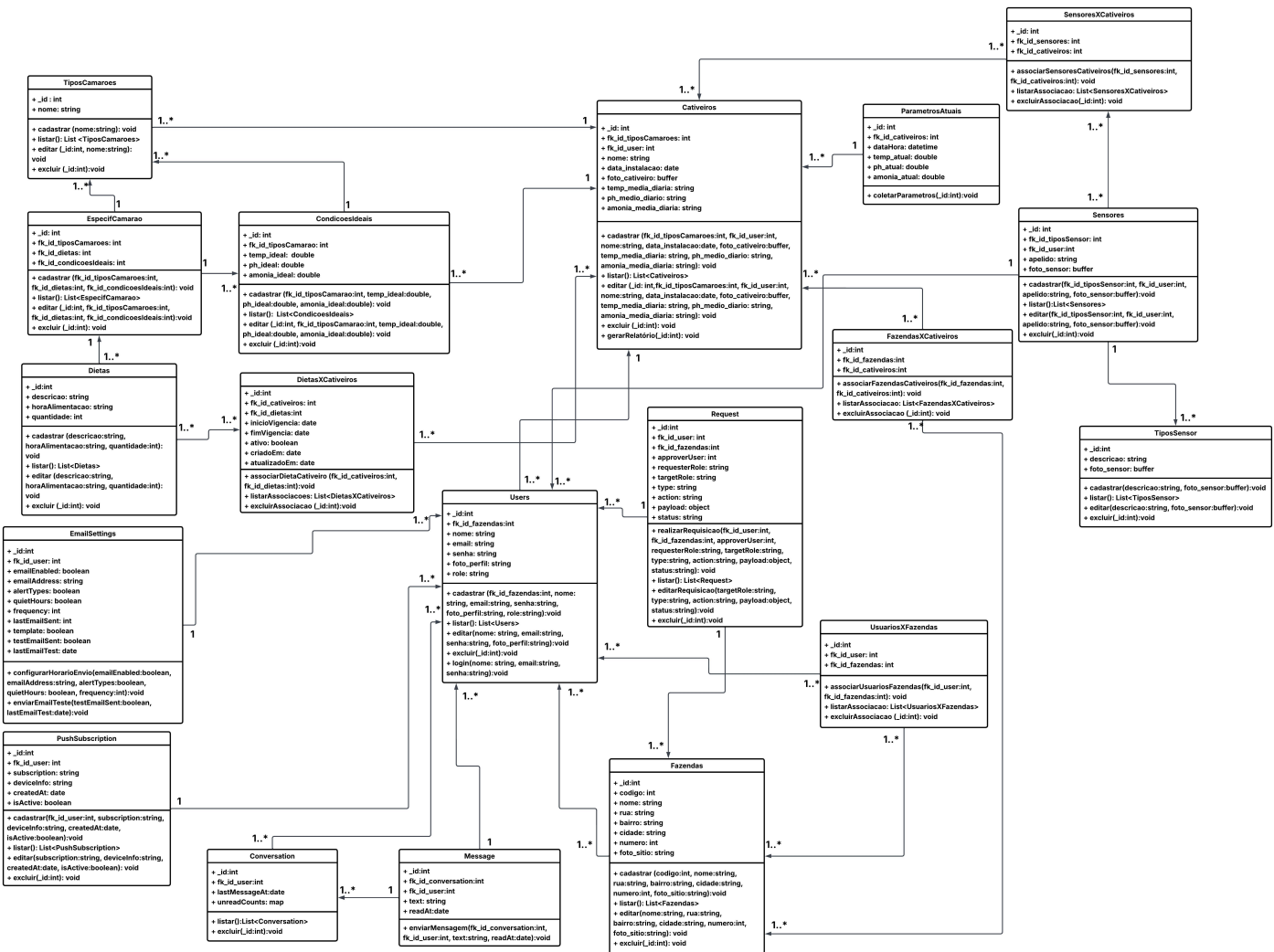
A classe `ParametrosAtuais` registra os parâmetros ambientais atuais de cada cativeiro, facilitando o monitoramento em tempo real dos dados de temperatura, pH e amônia. Os `SensoresXCativeiros` gerenciam a associação dos sensores aos cativeiros, garantindo uma coleta de dados precisa.

Além disso, classes como `Fazendas` e `Sítios` fornecem a infraestrutura física, podendo ter múltiplos cativeiros associados, enquanto classes de gerenciamento de usuários, como `Users`, cuidam do acesso ao sistema, incluindo funções de autenticação, gerenciamento de perfis e envio de notificações por email ou push.

As associações demonstram relacionamentos específicos: cada cativeiro está vinculado a um único tipo de camarão, mas uma espécie pode estar presente em múltiplos cativeiros; as dietas podem ser aplicadas a vários cativeiros ao longo do tempo; sensores monitoram condições específicas de cada cativeiro; e os pedidos de requisições gerenciam solicitações de usuários, incluindo aprovações e ações vinculadas ao sistema.

As multiplicidades (como 1..*, 0..1) reforçam a flexibilidade do sistema, possibilitando uma gestão robusta, com múltiplas dietas, sensores e cativeiros relacionados de forma consistente. Dessa forma, o sistema permite não apenas o monitoramento contínuo das condições ambientais, mas também a gestão eficiente das operações de criação de camarões, integrando dados, condições e ações em uma estrutura clara.

Figura 2 – Diagrama de classe



Fonte: Autoria Própria

DIAGRAMA DE OBJETOS

O diagrama de objetos apresenta uma visão prática do sistema de gerenciamento de cativeiros de camarões, mostrando instâncias concretas das classes e os valores associados em um momento específico. Ele representa o estado atual de cada entidade do sistema, suas relações e atributos, permitindo compreender de forma clara a configuração operacional.

Entre os objetos principais, temos o Cativeiro1, identificado como “Cativeiro principal”, que pertence ao usuário souzaMarcos09 e está associado ao tipo de camarão Rosa. Esse cativeiro registra médias diárias de temperatura (24,9°C), pH (6,2) e amônia (0,2), e está vinculado a sensores e parâmetros ambientais monitorados continuamente.

O Sensor1, apelidado de “Temp01”, está relacionado ao tipo de sensor “Sensor de temperatura” (TiposSensor1), e seus dados são armazenados em ParametrosAtuais1, com registros como temperatura atual de 25,4°C, pH 6,4 e amônia 0,1. Esses dados permitem avaliar se as condições do cativeiro estão próximas das CondiçõesIdeais1, que definem temperatura ideal de 25,4°C, pH ideal de 6,5 e amônia de 0,1.

A Dieta1, descrita como “Dieta à base de alface”, é aplicada ao cativeiro por meio da

relação DietasXCativeiros1, com início de vigência em 01/10/2025 e término em 10/01/2026, sendo registrada como ativa. Essa integração garante o controle alimentar dos camarões conforme o tipo e estágio de cultivo.

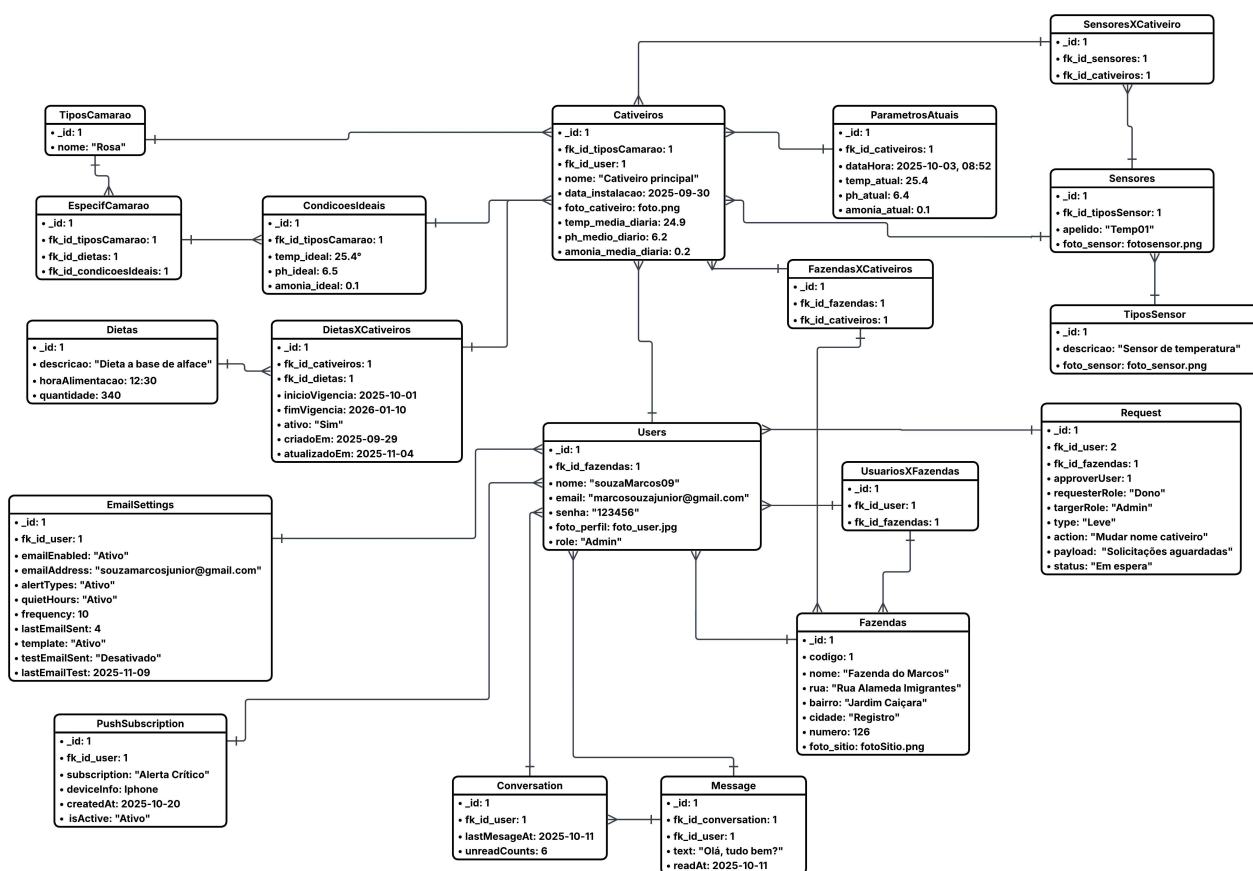
O sistema também gerencia as fazendas envolvidas na criação, como a Fazenda1, denominada “Fazenda do Marcos”, localizada na Rua Alameda Imigrantes, bairro Jardim Caiçara, cidade de Registro, com número 126. Essa fazenda é associada tanto ao usuário souzaMarcos09 quanto ao cativeiro em questão, estabelecendo o vínculo entre infraestrutura e operação.

O usuário souzaMarcos09, com papel de Admin, possui configurações personalizadas de e-mail (EmailSettings1) e notificações push (PushSubscription1), garantindo o recebimento de alertas e atualizações automáticas sobre o estado dos cativeiros. Ele também participa de conversas e mensagens no sistema, demonstradas pelos objetos Conversation1 e Message1, que armazenam interações como “Olá, tudo bem?”, com registro de leitura.

Além disso, o sistema gerencia solicitações administrativas por meio do objeto Request1, que representa um pedido de alteração no nome do cativeiro, feito pelo usuário souzaMarcos09 e com status “Em espera”, reforçando o controle de aprovação entre papéis como “Dono” e “Admin”.

Em conjunto, o diagrama evidencia como os objetos se inter-relacionam para permitir um controle inteligente e integrado da criação de camarões. Sensores coletam dados ambientais, dietas são aplicadas de forma planejada, e o sistema fornece notificações e registros de comunicação entre usuários e fazendas. Essa integração assegura o monitoramento eficiente das condições de cultivo e o bem-estar dos camarões.

Figura 3 – Diagrama de objetos



Fonte: Autoria Própria

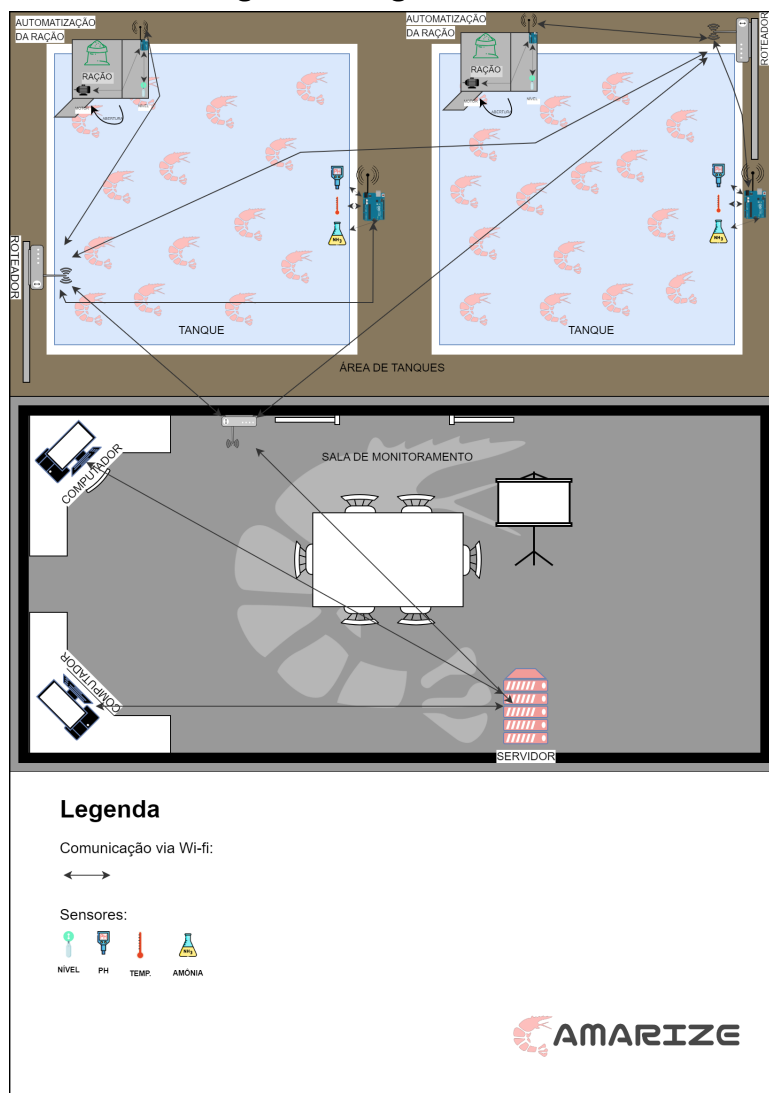
DIAGRAMA DE REDES

O Diagrama de Redes foi desenvolvido com o objetivo de ilustrar o funcionamento do projeto Camarize, um Sistema de Monitoramento de Cativeiro de Camarões para Fins Gastronômicos.

Na etapa de monitoramento, cada tanque estará equipado com um Arduino conectado a três sensores: medidor de temperatura, de pH e amônia. Que se mostram fundamentais para a avaliação da salubridade da água do cativeiro para os camarões. O dispositivo transmitirá continuamente os dados dos sensores por meio do Wi-Fi para o servidor central encontrado na sala de monitoramento.

Na etapa de automatização da alimentação, será utilizado um dispensador de ração com uma abertura na parte inferior, equipado com um motor para controle da abertura e um sensor de nível com os parâmetros pré-definidos anteriormente para determinar a quantidade ideal de ração a ser liberada no tanque. Em que, ambos os componentes estarão conectados ao Arduino, também se comunicando via Wi-Fi com o servidor central da sala de monitoramento. Os dados serão então compartilhados com os computadores que têm acesso à plataforma.

Figura 4 – Diagrama de Rede



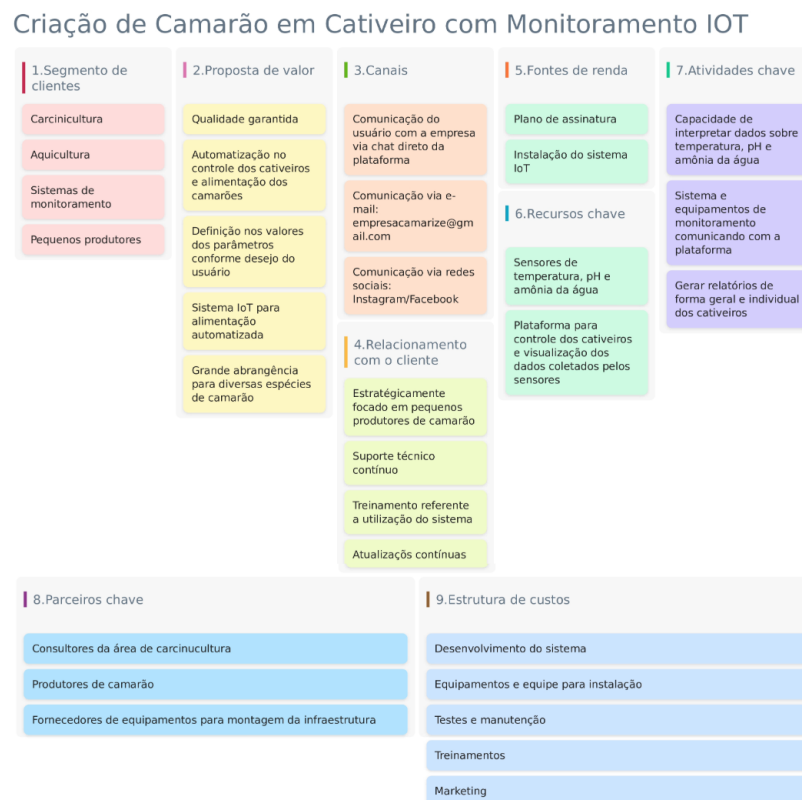
Fonte: Autoria Própria

É essencial pontuar que a comunicação entre os elementos ocorrerá via Wireless, que pode ser amplificada com uma maior quantidade de roteadores a depender da localização.

MODELO DE NEGÓCIOS CANVAS

O modelo de negócios Canvas foi utilizado para análise econômica do sistema. Por meio do Canvas foi possível identificar os Parceiros, Recursos e Atividades Chaves, nossas Propostas de Valor, Segmento de Mercado, nossa Fonte de Renda e Estrutura de Custos.

Figura 5 – Canvas



Fonte: Autoria Própria

Foi realizada uma análise completa e incluído os possíveis dados.

MODELO LÓGICO E CONCEITUAL DO BANCO DE DADOS

A modelagem conceitual e lógica demonstra a funcionalidade do programa e do que será aplicado no banco de dados. A visão gráfica do banco de dados se demonstra algo fundamental, por conta que proporciona o máximo de detalhes com extrema clareza, apresentando as relações entre as entidades.

No modelo conceitual, a entidade *Cativeiros* representa os tanques de criação, registrando informações essenciais sobre o ambiente de cultivo, como data de instalação, temperatura média diária, pH médio diário e amônia média diária. Cada cativeiro está associado a um Tipo de Camarão, definido na entidade *TiposCamarao*, que identifica a espécie cultivada, e a *CondiçõesIdeais*, responsável por armazenar os parâmetros ambientais recomendados, incluindo temperatura ideal, pH ideal e amônia ideal, servindo como referência para avaliação das condições reais do ambiente.

O monitoramento é realizado por meio da entidade *Sensores*, que registra dispositivos responsáveis pela coleta de dados em tempo real, como temperatura, pH e níveis de amônia. Cada sensor está associado a um tipo específico, definido na entidade *TiposSensor*, e suas medições são registradas em *ParametrosAtuais*, com atributos como *dataHora*, temperatura atual, pH atual e amônia atual. Essa estrutura possibilita a comparação contínua entre valores medidos e parâmetros ideais, contribuindo para a manutenção das condições adequadas ao cultivo.

O controle nutricional é modelado pelas entidades *Dietas* e *DietasXCativeiros*, que definem as composições alimentares e seus períodos de aplicação. A entidade *Dietas* armazena dados como descrição, horário de alimentação e quantidade, enquanto *DietasXCativeiros* relaciona essas dietas aos respectivos tanques, incluindo informações sobre vigência, data de criação e atualização, permitindo um acompanhamento preciso da alimentação dos camarões ao longo do tempo.

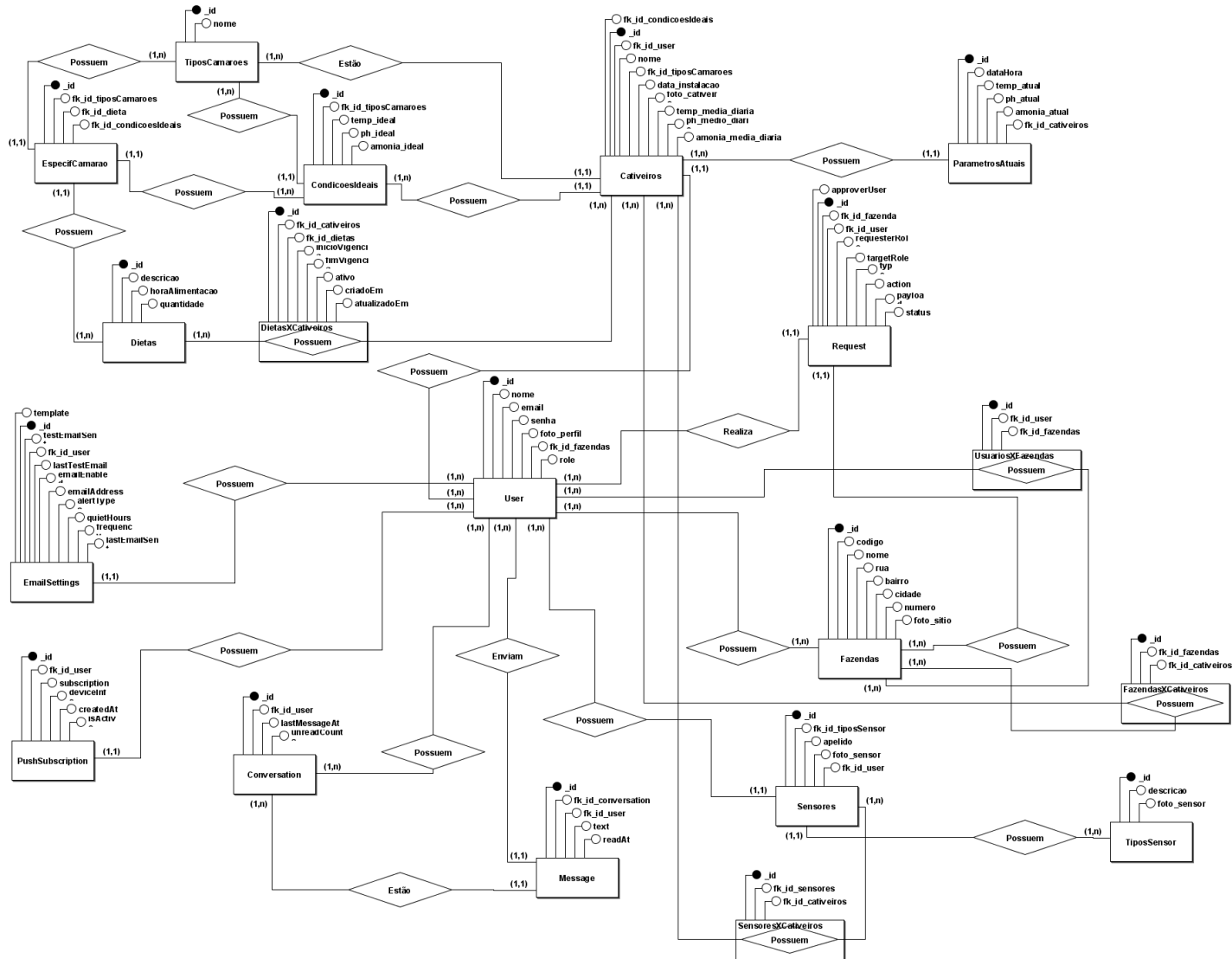
As Fazendas representam as unidades produtivas, contendo informações como nome, endereço, bairro, cidade, número e foto do local, e estão associadas a múltiplos cativeiros por meio da entidade *FazendasXCativeiros*. Essa relação reflete a organização física e operacional da produção aquícola, permitindo a integração de diversos tanques sob uma mesma gestão.

A entidade *Users* descreve os usuários do sistema, armazenando dados como nome, e-mail, senha, função (role) e foto de perfil. Os usuários podem estar vinculados a uma ou mais fazendas por meio da entidade *UsuariosXFazendas*, e possuem configurações adicionais em *EmailSettings* e *PushSubscription*, que definem preferências de notificação, frequência de alertas e dispositivos de recebimento. Além disso, as entidades *Conversation* e *Message* possibilitam a comunicação interna entre usuários, registrando interações e mensagens trocadas no sistema.

O modelo também contempla a entidade *Request*, responsável por gerenciar solicitações administrativas, como alterações de nome de cativeiro ou permissões de acesso, com controle de aprovação, papéis de solicitante e aprovador, e status das requisições.

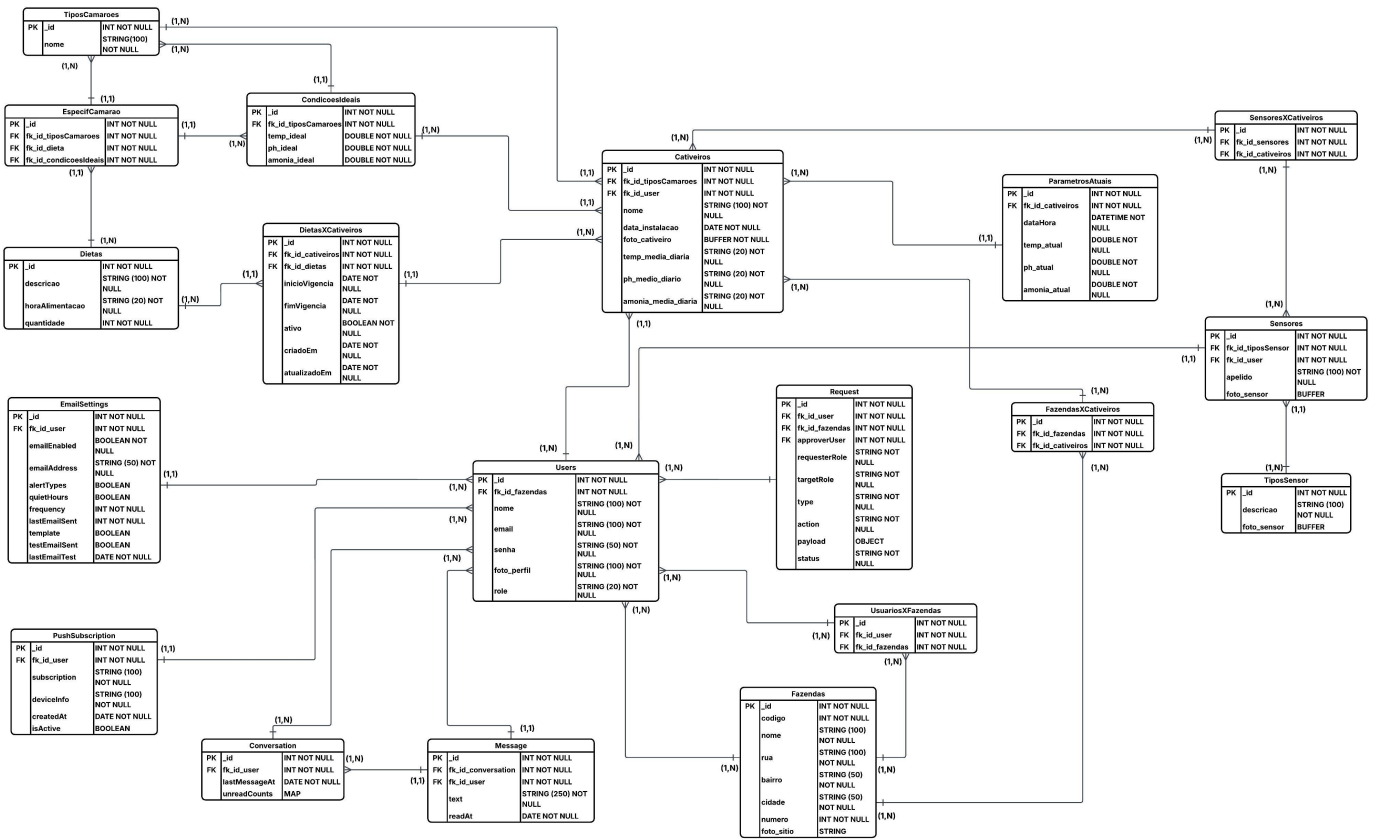
No modelo lógico, é apresentado uma representação mais concreta do banco de dados, demonstrando as tabelas, entidades, chaves primárias e estrangeiras e seus relacionamentos.

Figura 6 – Modelo Conceitual



Fonte: Autoria Própria

Figura 7 – Modelo Lógico



Fonte: Autoria Própria

O banco de dados tem um papel de extrema importância para as informações, sendo necessário para guardar, organizar e recuperar dados. O Modelo Lógico e Conceitual são maneira de demonstrar a eficiência de um banco de dados.

ANÁLISE DE RECORRÊNCIA DO ALGORITMO MERGE SORT

O Merge Sort é um algoritmo de ordenação que consiste em dividir uma estrutura de subconjuntos e aplicar a ordenação nos elementos que são extraídos da estrutura primária. Após a ordenação dos subconjuntos, é realizada a mistura para um conjunto final ordenado.

O melhor tempo ocorre quando todos os elementos do array já estão ordenados, o Merge Sort sempre executa a divisão e a fusão, independente da ordem dos elementos. O tempo de execução, representado por $O(n \log_2 n)$, pode ser explicado da seguinte forma: $\log_2 n$ corresponde ao número de divisões do array até que cada subarray contenha apenas um elemento, e n refere-se ao número de comparações e fusões realizadas durante o processo de merge.

O tempo médio ocorre quando os elementos estão em ordens aleatórias, em seu comportamento o número de divisões e fusões não dependem da ordem dos elementos, cada nível realiza n operações no total e possui $\log_2 n$ níveis. Sua representação é semelhante com o melhor tempo: $O(n \log_2 n)$.

O pior tempo ocorre quando o array está em ordem oposta a esperada, sua complexidade não é alterada por conta que sua quantidade de execuções não são alteradas. Sua representação se mantém a mesma, sendo ela: $\log_2 n$.

Na aplicação, foi utilizada na tela de sensores, sendo utilizado para organizar os sensores de forma crescente e decrescente. Seu tempo de execução foram exatos 0.000000 milissegundos, mostrando a eficiência e rapidez do algoritmo. Sua representação matemática se apresenta desta maneira:

$$\text{MergeSort}(n) = \begin{cases} 0, & \text{se } n = 0 \\ 0, & \text{se } n = 1 \\ 2 \cdot \text{MergeSort}\left(\frac{n}{2}\right) + O(n), & \text{se } n > 1 \end{cases}$$

A representação do código:

```
function mergeSort(arr) {
  if (arr.length <= 1) return arr;

  const mid = Math.floor(arr.length / 2);
  const left = mergeSort(arr.slice(0, mid));
  const right = mergeSort(arr.slice(mid));

  return merge(left, right);
}

function merge(left, right) {
  let result = [], leftIndex = 0, rightIndex = 0;

  while (leftIndex < left.length && rightIndex < right.length) {
    if (left[leftIndex].id_tipo_sensor > right[rightIndex].id_tipo_sensor) {
      result.push(left[leftIndex]);
      leftIndex++;
    } else {
      result.push(right[rightIndex]);
    }
  }
}
```

```

        rightIndex++;
    }
}

return result.concat(left.slice(leftIndex), right.slice(rightIndex));
}

```

A representação em pseudocódigo:

```

funcao mergeSort(arr[])

    // Função recursiva para ordenar o vetor usando o algoritmo MergeSort

    se tamanho de arr <= 1 então
        retornar arr
    fimse

    meio <- arredondar_para_baixo(arr.length / 2)
    esquerda <- mergeSort(arr[0, meio])
    direita <- mergeSort(arr[meio, arr.length])

    retornar merge(esquerda, direita)
fimfuncao

funcao merge(esquerda[], direita[])

    // Função para combinar dois vetores ordenados

    resultado <- lista_vazia
    indice_esquerda <- 0
    indice_direita <- 0

    enquanto indice_esquerda < tamanho de esquerda e
    indice_direita < tamanho de direita faça
    se
        esquerda[indice_esquerda].id_tipo_sensor > direita[indice_direita].id_tipo_sensor
            então
                adicionar esquerda[indice_esquerda] em resultado
                indice_esquerda incrementa 1
            senão
                adicionar direita[indice_direita] em resultado
                indice_direita incrementa 1
        fimse
    fimenquanto

```

```
    retornar resultado concatenado com esquerda[indice_esquerda, arr.length]
    e direita[indice_direita, arr.length]
fimfuncao
```

APLICAÇÃO IOT

O código foi desenvolvido para um ESP32 que monitora parâmetros ambientais de um cativeiro aquático, como temperatura, pH e amônia, por meio de sensores analógicos conectados aos pinos ADC1 do microcontrolador. As leituras são processadas e, no caso da temperatura, convertidas de voltagem para graus Celsius com base em uma fórmula específica. As informações são exibidas em tempo real na aplicação mobile em um dashboard, permitindo o acompanhamento visual constante das condições do ambiente.

Além do monitoramento, o sistema inclui um mecanismo de alimentação automática controlado por um servo motor. A cada intervalo de tempo definido por padrão, a cada 30 segundos, o servo realiza um número de giros programado para simular a liberação de ração. Esses ciclos de alimentação podem ser ajustados dinamicamente pelo usuário através do monitor serial, oferecendo flexibilidade para diferentes necessidades de manejo. Dessa forma, o código fornece uma solução prática e automatizada para o controle de alimentação e monitoramento básico da qualidade da água em ambientes de criação.

```
#include <WiFi.h>
#include <NTPClient.h>
#include <WiFiUdp.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <ESP32Servo.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <Preferences.h>
#include <WiFiClientSecure.h>

// Configurações WiFi
const char* ssid = "JOÃO VITOR";
const char* password = "cs27062005";

// Configurações da API
const char* apiBaseUrl = "https://app-0-2-camarize-v2.onrender.com";

// Armazenamento do cativeiroId (persistido em NVS)
Preferences prefs;
String cativeiroId; // ObjectId como string

// NTP Client
WiFiUDP ntpUDP;
NTPClient timeClient(ntpUDP, "pool.ntp.org", -10800);
```

```

// Pinos dos sensores
const int pHPin = 32;
const int amoniaPin = 35;
const int tempPin = 26;
OneWire oneWire(tempPin);
DallasTemperature sensors(&oneWire);

// Servo para alimentação
Servo servo360;
const int servoPin = 25;

// Variáveis de controle
int horaAlvo = 17;
int minutoAlvo = 0;
float racaoDesejada = 25.0;
const float racaoPorGiro = 10.0;
String ultimaDataGiro = "";

// Controle de timing
unsigned long ultimoFetch = 0;
const unsigned long intervaloFetch = 30000; // 30 segundos

void setup() {
  Serial.begin(9600);
  analogSetAttenuation(ADC_11db);

  Serial.println("Conectando ao WiFi...");
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("\nWiFi conectado!");
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());

  // Inicializar NTP
  timeClient.begin();

  // Inicializar sensores
  sensors.begin();

  // Inicializar servo

```

```

servo360.attach(servoPin);
servo360.write(90);

// Inicializar NVS e obter cativoId
prefs.begin("camarize", false);
cativoId = prefs.getString("cativId", "");

// SET CATIVEIRO <ObjectId>
Serial.println("=== COMANDOS DISPONÍVEIS ===");
Serial.println("SET CATIVEIRO <ObjectId> - Configurar/alterar cativo");
Serial.println("CLEAR CATIVEIRO - Limpar cativo atual");
Serial.println("SHOW CATIVEIRO - Mostrar cativo atual");
Serial.println("HELP - Mostrar comandos");
Serial.println("=====");

if (!cativoId.isEmpty()) {
    Serial.println("Cativo atual: " + cativoId);
} else {
    Serial.println("Nenhum cativo configurado");
}

Serial.println("Digite: SET CATIVEIRO <ObjectId> para configurar (ou aguarde)");

unsigned long start = millis();
while (cativoId.isEmpty() && millis() - start < 10000) {
    if (Serial.available()) {
        String line = Serial.readStringUntil('\n');
        line.trim();
        line.toUpperCase();

        if (line.startsWith("SET CATIVEIRO ")) {
            String val = line.substring(14);
            val.trim();

            if (val.length() >= 24) {
                cativoId = val;
                prefs.putString("cativId", cativoId);
                Serial.println("Cativo configurado: " + cativoId);
            } else {
                Serial.println("ObjectId inválido (deve ter 24+ caracteres)");
            }
        } else if (line == "CLEAR CATIVEIRO") {
            cativoId = "";
            prefs.remove("cativId");
            Serial.println("Cativo removido");
        } else if (line == "SHOW CATIVEIRO") {

```



```

        Serial.println("Cativeiro atual: " + cativeiroId);
    } else if (line == "HELP") {
        Serial.println("=== COMANDOS DISPONÍVEIS ===");
        Serial.println("SET CATIVEIRO <ObjectId> - Configurar/alterar cativeiro");
        Serial.println("CLEAR CATIVEIRO - Limpar cativeiro");
        Serial.println("SHOW CATIVEIRO - Mostrar cativeiro atual");
        Serial.println("HELP - Mostrar comandos");
        Serial.println("=====");
    }
}
}
delay(50);
}

if (cativeiroId.isEmpty()) {
    Serial.println("Cativeiro não configurado. Use 'SET CATIVEIRO <ObjectId>'
    durante operação");
}

// Buscar dieta inicial
buscarDietaDaAPI();
}

void loop() {
    // Verificar comandos durante operação
    if (Serial.available()) {
        String line = Serial.readStringUntil('\n');
        line.trim();
        line.toUpperCase();

        if (line.startsWith("SET CATIVEIRO ")) {
            String val = line.substring(14);
            val.trim();

            if (val.length() >= 24) {
                cativeiroId = val;
                prefs.putString("cativId", cativeiroId);
                Serial.println("Cativeiro alterado para: " + cativeiroId);
                // Buscar nova dieta do novo cativeiro
                buscarDietaDaAPI();
            } else {
                Serial.println("ObjectId inválido (deve ter 24+ caracteres)");
            }
        }

        } else if (line == "CLEAR CATIVEIRO") {
            cativeiroId = "";
            prefs.remove("cativId");
        }
    }
}

```

```

        Serial.println("Cativeiro removido");

    } else if (line == "SHOW CATIVEIRO") {
        Serial.println("Cativeiro atual: " + cativeiroId);

    } else if (line == "HELP") {
        Serial.println("=== COMANDOS DISPONÍVEIS ===");
        Serial.println("SET CATIVEIRO <ObjectId> - Alterar cativeiro");
        Serial.println("CLEAR CATIVEIRO - Limpar cativeiro");
        Serial.println("SHOW CATIVEIRO - Mostrar cativeiro atual");
        Serial.println("HELP - Mostrar comandos");
        Serial.println("=====");
    }
}

timeClient.update();
unsigned long agora = millis();

if (agora - ultimoFetch >= intervaloFetch) {
    ultimoFetch = agora;

    // Buscar dieta atualizada
    buscarDietaDaAPI();

    // Obter dados atuais
    int currentHour = timeClient.getHours();
    int currentMinute = timeClient.getMinutes();
    time_t epochTime = timeClient.getEpochTime();
    struct tm* timeinfo = localtime(&epochTime);

    char dataAtual[11];
    sprintf(dataAtual, "%04d-%02d-%02d", 1900 + timeinfo->tm_year, 1 + timeinfo->tm_mon, timeinfo->tm_mday);
    String dataHoje = String(dataAtual);
    String dataHoraISO = dataHoje + "T" + timeClient.getFormattedTime();

    // Ler sensores
    int pHValue = analogRead(pHPin);
    float voltagePH = (pHValue / 4095.0) * 3.3;
    float pH = 7 + ((2.5 - voltagePH) / 0.18);
    pH = 7.2; // Valor fixo para teste

    int amoniaValue = analogRead(amoniaPin);
    float voltageAmonia_medida = (amoniaValue / 4095.0) * 3.3;
    float voltageAmonia = voltageAmonia_medida * 31.3;
    float amonia = voltageAmonia * 10;

```

```

sensors.requestTemperatures();
float temperatureC = sensors.getTempCByIndex(0);

// Log dos dados
Serial.printf("Hora: %02d:%02d | Data: %s | pH: %.2f | Amônia: %.2f ppm
| Temp: %.2f °C\n",currentHour, currentMinute, dataHoje.c_str(),
    pH, amonia, temperatureC);

// Enviar dados para API
enviarParametrosParaAPI(temperatureC, pH, amonia, cativoId, dataHoraISO);

// Verificar se é hora de alimentar
int girosNecessarios = (int)((racaoDesejada / racaoPorGiro) + 0.5);

if (currentHour == horaAlvo && currentMinute == minutoAlvo && ultimaDataGiro
!= dataHoje) {
    Serial.printf("Liberando %.2f g de ração (%d giros).\n",
        racaoDesejada, girosNecessarios);

    for (int i = 0; i < girosNecessarios; i++) {
        Serial.printf("Giro %d/%d...\n", i + 1, girosNecessarios);
        servo360.write(180);
        delay(2000);
        servo360.write(90);
        delay(500);
    }

    Serial.println("Giros completos.");
    ultimaDataGiro = dataHoje;
}
}

delay(100);
}

// Função para buscar dieta da API
void buscarDietaDaAPI() {
    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        WiFiClientSecure client;
        client.setInsecure(); // Permitir HTTPS sem verificação de certificado

        if (cativoId.length() < 24) {
            Serial.println("cativoId inválido/não configurado");
            return;
        }
    }
}

```

```

}

String url = String(apiBaseUrl) + "/dietas/atual/" + cativeiroId;
http.begin(client, url); // Usar cliente seguro

int httpResponseCode = http.GET();

if (httpResponseCode == 200) {
    String payload = http.getString();
    Serial.print("JSON da dieta recebida: ");
    Serial.println(payload);

    StaticJsonDocument<200> doc;
    DeserializationError error = deserializeJson(doc, payload);

    if (!error) {
        // Verificar se tem hora de alimentação
        if (doc.containsKey("horaAlimentacao") && doc["horaAlimentacao"] != nullptr) {
            const char* horaAlimentacao = doc["horaAlimentacao"];
            char horaCortada[6] = {0};
            strncpy(horaCortada, horaAlimentacao, 5);

            int h = 0, m = 0;
            if (sscanf(horaCortada, "%d:%d", &h, &m) == 2) {
                horaAlvo = h;
                minutoAlvo = m;
                Serial.printf("Hora de alimentação atualizada: %02d:%02d\n",
                    horaAlvo, minutoAlvo);
            } else {
                Serial.println("Erro ao parsear horaAlimentacao");
            }
        }

        // Atualizar quantidade
        if (doc.containsKey("quantidade")) {
            racaoDesejada = doc["quantidade"];
            Serial.printf("Quantidade atualizada: %.2f g\n", racaoDesejada);
        }

        // Log da descrição da dieta
        if (doc.containsKey("descricao")) {
            const char* descricao = doc["descricao"];
            Serial.printf("Dieta: %s\n", descricao);
        }

        Serial.printf("Dieta atualizada: %02d:%02d - %.2f g\n", horaAlvo,

```

```

        minutoAlvo, racaoDesejada);
    } else {
        Serial.println("Erro ao parsear JSON dieta");
    }
} else {
    Serial.printf("Erro HTTP ao buscar dieta: %d\n", httpResponseCode);
}

http.end();
} else {
    Serial.println("WiFi desconectado");
}
}

// Função para enviar parâmetros para API
void enviarParametrosParaAPI(float temp, float ph, float amonia,
String idCativeiro, String dataHoraISO) {
    Serial.println("=== DEBUG ENVIAR PARAMETROS ===");
    Serial.println("WiFi Status: " + String(WiFi.status()));
    Serial.println("WiFi IP: " + WiFi.localIP().toString());

    if (WiFi.status() == WL_CONNECTED) {
        HTTPClient http;
        WiFiClientSecure client;
        client.setInsecure(); // Para HTTPS

        String url = String(apiBaseUrl) + "/parametros/cadastrar";

        Serial.println("URL completa: " + url);
        Serial.println("cativeiroId: " + idCativeiro);
        Serial.println("cativeiroId length: " + String(idCativeiro.length()));

        http.begin(client, url); // Usar client seguro
        http.addHeader("Content-Type", "application/json");

        if (idCativeiro.length() < 24) {
            Serial.println("cativeiroId inválido/não configurado");
            http.end();
            return;
        }

        String json = "{";
        json += "\"id_cativeiro\": \"" + idCativeiro + "\", ";
        json += "\"temperatura\": " + String(temp, 2) + ", ";
        json += "\"ph\": " + String(ph, 2) + ", ";
        json += "\"amonia\": " + String(amonia, 2);
    }
}

```

```

    json += "}";

    Serial.println("JSON enviado: " + json);

    int httpResponseCode = http.POST(json);

    Serial.println("Response Code: " + String(httpResponseCode));

    if (httpResponseCode > 0) {
        String response = http.getString();
        Serial.println("Sucesso:");
        Serial.println(response);
    } else {
        Serial.println("Erro HTTP: " + String(httpResponseCode));
        Serial.println("Erro descrição: " + http.errorToString(httpResponseCode));
    }

    http.end();
} else {
    Serial.println("WiFi desconectado");
}

Serial.println("=== FIM DEBUG ===");
}

```

ANÁLISE SWOT

A análise SWOT é uma ferramenta estratégica que avalia os Fatores Positivos (Forças e Oportunidades) e os Fatores Negativos (Fraquezas e Ameaças) de um sistema. Essa análise é fundamental para entender o potencial e os desafios do projeto.

DIÁRIO DE BORDO

Nome da Atividade	Data de início	Data de término	Responsável pela atividade	Descrição da atividade realizada
Início dos trabalhos para o projeto	06/08/2025	13/08/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto;	Início dos planejamentos e realizações para o projeto no 4º Semestre
Modificações na aplicação	06/08/2025	29/10/2025	João Kusaka;	Processo de alterações na aplicações, dentre as modificações estão: Alteração do código para utilizar React e hospedagem do banco de dados na nuvem no Mongo Atlas.
Reunião do P.I	11/08/2025	11/08/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto;	Reunião realizada para organizar as ideias do projeto, realizar mudanças em relação ao funcionamento do projeto e definir tarefas para cada integrante.
Modificações no artigo científico	11/08/2025	29/10/2025	Tiago Rodrigues;	Alterações realizadas no artigo para melhorias de escrita e inclusão de mais informações.

Criação do questionário para a pesquisa de campo 1º versão	13/08/2025	28/08/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto;	Formulação da primeira versão do questionário para a pesquisa de campo com auxílio da professora Simone.
Modificações no figma	14/08/2025	29/10/2025	Matheus Ferrari.	Modificações feitas em todas as telas com base nas alterações feitas na aplicação web.
Inscrição para o XXXVII Congresso da UNESP de Iniciação Científica	29/08/2025	03/09/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto	Fomos selecionados para se inscrever no Congresso da UNESP, o depósito dos requisitos foi realizado por Tiago Rodrigues.
Criação do questionário para pesquisa de campo 2º versão	04/09/2025	05/09/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto;	Com base nas correções sugeridas feitas pelo professor Frederico e professora Simone, foi feita a revisão e alterações nas questões criando assim a segunda versão do questionário.
Modificação no arduíno	29/08/2025	29/10/2025	Leandro Augusto;	Alterações no código do arduíno e no modelo 3D do projeto para melhorar a performance e ser de fácil uso.

Visita na UNESP Registro para pesquisa de campo	15/09/2025	15/09/2025	Tiago Rodrigues; Victor Roder; Leandro Augusto.	Realizada a primeira pesquisa de campo na UNESP Registro. Foi conduzida pela professora Giovana Bertini e pela aluna Estephany Konesuk
Reunião do P.I	17/09/2025	17/09/2025	João Kusaka; Matheus Abrahão; Tiago Rodrigues; Victor Roder; Leandro Augusto;	Realizada uma reunião na FATEC para adição de uma nova funcionalidade no sistema, para organizar pontos importantes e sobre o andamento do projeto.
Inclusão da funcionalidade de tutorial	18/09/2025	20/09/2025	João Kusaka.	Realizada a inclusão da funcionalidade de tutorial para facilitar a utilização para o usuário. A funcionalidade foi decidida na aula de Experiência de Usuário.
Coleta de camarões	22/09/2025	22/09/2025	Leandro Muniz.	Feita a coleta dos camarões que foram doados para o projeto pela professora Giovana Bertini da UNESP Registro
Refazimento dos diagramas	29/09/2025	29/10/2025	Tiago Rodrigues.	Realizada as alterações nos diagramas do projeto com base em todas as mudanças realizadas no 4º semestre

Workshop sobre o artigo	10/10/2025	10/10/2025	Tiago Rodrigues.	Workshop realizado pelo professor Frederico para auxiliar nas melhoras dos artigos e dicas.
Modificações no Banner	13/10/2025	15/10/2025	Tiago Rodrigues.	Foram realizadas mudanças no banner para estarem com as informações atualizadas acerca do projeto e dos novos protótipos de tela. Banner atualizado pelo Figma.
Modificações no Pitch	16/10/2025	29/10/2025	João Kusaka; Leandro Augusto; Matheus Ferrari; Tiago Rodrigues	Foram realizadas alterações no pitch com base nos requisitos do MindMap